

Data Model → collection of concepts that can be used to define the structure of a db which necessarily involves abstraction

data types relation ships constraints

also include a set of basic operations

Nowadays:
user-defined ops. retrieval updates.

dynamic aspect / behaviour.

(U-2)

Categories of Data Model

1) High Level / Conceptual DM

• entities, attributes, relationships.

• Eg: ER Model

• deal with properties that describe an entity.

• represents association among entities.

2) Low Level / Physical DM

• how data is stored on computer storage media

→ Eg: Magnetic disks

• Eg: As files by representing info. as record formats, record orderings & access paths.

↳ a structure that makes database record search efficient

Eg: Index (Indices can be organized in linear, hierarchical / some other fashion).

3) Representation / Implementation DM

• lies between low and high level DM.

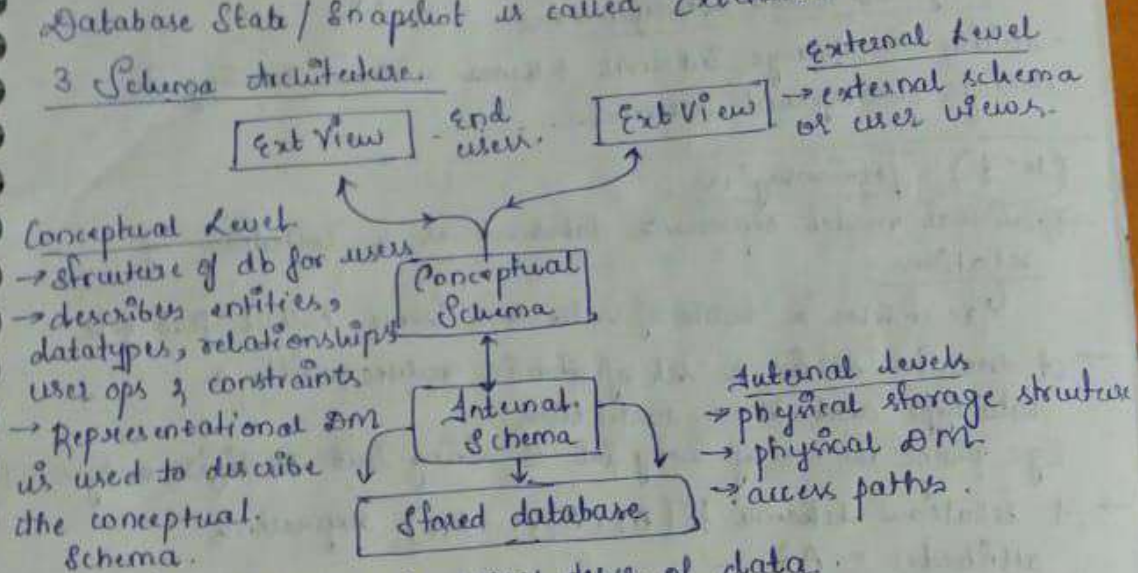
• used in traditional commercial DBMS for implementing on a computer system

• Eg: Relational DM, Network DM, Hierarchical DM

• Representational DM use record structures to represent data & hence are called Record based DM

- In a DB. $\rightarrow$ descr. of database = Database Schema
 $\hookrightarrow$ each object in the schema = Schema Construct.
- data in the db at a particular moment in time is called Database State / Snapshot / Current set of Occurrences or Instances in the db.
- Schema is sometimes called Antevision, whereas Database State / Snapshot is called Extension.

3 Schema Architecture.



Note: The 3 schemas are only descr. of data.
 Stored data exists in physical level only.

* The processes of transforming requests from ext. view and results from stored db or between levels based on Schema format / reformat is called Mappings.

Data Independence.

↳ the ability to change the schema at one level of dbms without having to change schema at next higher level.

1) Logical Data Independence

• ability to change conceptual schema without having to change external schema / application pgms.

2) Physical Data Independence

• ability to change internal schema without having to change conceptual schema.

(Ch-3) Terminologies

→ Relational model represents database as a collection of relations.

↳ resembles a table of values & hence called flat file.

→ A domain D is a set of atomic values with a data type and some restriction.

Eg: phone no. is of long int or string but to digits only.

→ A relation schema $R(A_1, A_2, \dots, A_n)$ represents attributes = A_i

D is the domain of $A_i = \text{dom}(A_i)$

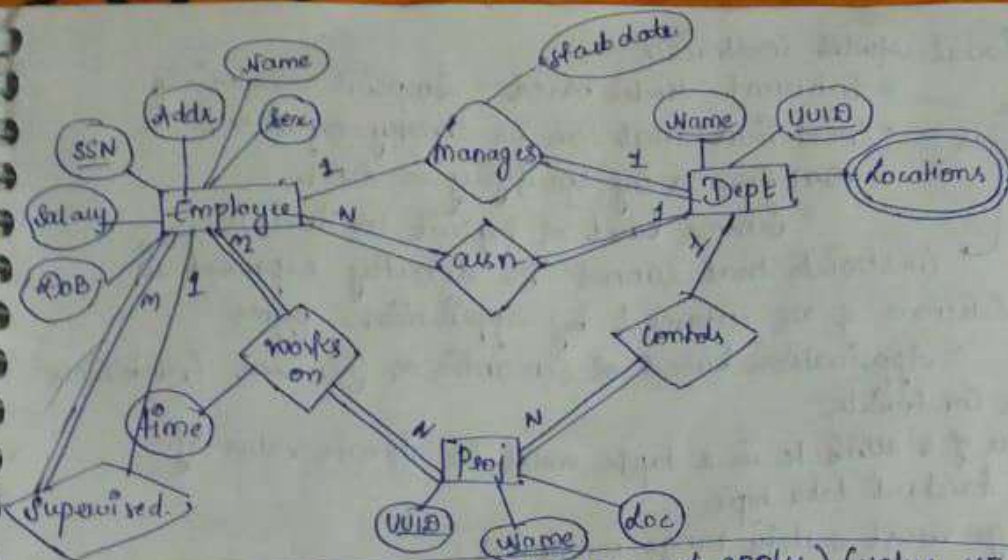
degree or arity of a relation R = no. of attributes

→ A relation (or relation state) denoted as $r(R)$ is a set of n -tuples $r = \{t_1, t_2, \dots, t_m\}$ where a tuple

$t_i = \text{ordered list } \langle v_1, v_2, \dots, v_n \rangle$ where v_k represents value of attribute A_k in the row i .

→ Relation Intension = Relation Schema = R

Relation extension = Relation state = $r(R)$.



NULL $\rightarrow$ not applicable (attribute does not apply) (value undefined)
 $\rightarrow$ exists, but missing (value exist but unavail.)
 $\rightarrow$ unknown $\rightarrow$ not known (value unknown).

$\rightarrow$ Using Set theory, relation r can be redefined as:

$r(R)$ is a mathematical relation of degree n on the domains $\text{dom}(A_1), \text{dom}(A_2), \dots, \text{dom}(A_n)$ which is a subset of the Cartesian product

$$r(R) \subseteq (\text{dom}(A_1) \times \text{dom}(A_2) \times \dots \times \text{dom}(A_n))$$

$\rightarrow$ Total no. of possible tuples in $r(R) = |D_1| \times |D_2| \times \dots \times |D_n|$

where $|D_i| = \text{cardinality / degree of } A_i$

$\rightarrow$ There is no ordering of Tuples in a relation.

$$r = \{t_1, t_2, \dots, t_m\} = \{t_2, t_1, \dots, t_m\}$$

$\rightarrow$ But each tuple is an ordered n -dimensional list.

$\rightarrow$ Each value in a tuple is an Atomic value and hence composite & multivalued attributes are not allowed.

Hence it's called Flat relational model with first normal form.

Relational Model Constraints

- 3 main categories →
- Ambient model based = Implicit constraints
 - Constraints that can be directly expressed in DB Schema by specifying in DDL
= Schema based or Explicit constraints.
 - Constraints that cannot be directly expressed in Schemas & are enforced by application pgms
= Application-based or Semantic or Business Constraints

1) Domain Constraints

- value of a attribute in a tuple must be atomic value of a standard data type
- range must satisfy range constraint

2) Key Constraints

- No 2 tuples can have all values same $\therefore X$ is a set
- Let SK = subset of some attributes
then $\forall t_1, t_2 \in X(R)$ if $t_1[SK] \neq t_2[SK]$
then SK = Superkey of relation schema R
- Every relation has atleast 1 SK which is = Set of all attributes
- 1 primary key and other candidate keys are called unique (unique) Keys

We know relational schema and relational state.

Similarly,

- Relational database schema $S = \{R_1, R_2, \dots, R_m\}$ and set of Integrity Constraints
- Relational database state $DB = \{x_1, x_2, \dots, x_m\}$
where x_i is a state of R_i , i.e. $x_i \in R_i$

3) Entity Integrity Constraint

- primary key cannot be NULL.

4) Referential Integrity Constraint

- tuple in one relation must refer to an existing tuple in another relation.

A set of attributes FK in relation schema R_1 is a foreign key of R_1 that references relation R_2 if:

→ $\text{dom}(FK \text{ in } R_1) = \text{dom}(PK \text{ in } R_2)$.

→ For $t_1 \in R_1$ and $t_2 \in R_2$, $t_1[FK] = t_2[PK]$.

R_1 is called referencing relation.

R_2 is called referenced relation.

5) Other types of constraints:

→ Constraints that a valid state of db must satisfy
= State constraints (as discussed previously).

→ Constraints that deal with state changes
= Transition constraints (enforced by app. pgms.)

Transactions & Constraint Violations

1) Insert Operation.

- Domain const
- Key const
- Entity Integrity
- Referential Integ.

2) Delete Op.

- referential Integrity
- Solution
 - Cascade
 - Restrict
 - Set Null or set default.

3) Update Op.

All +.